

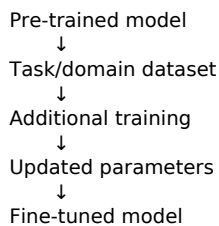
FINE-TUNING

From Scratch → Deep Concepts → Practical Workflow → Interview Preparation

Purpose: This guide is designed to make fine-tuning understandable from first principles and to prepare you for ML/AI/LLM interviews. It covers what fine-tuning is, why it is used, training mechanics, datasets, tokenization, loss, backpropagation, optimizers, hyperparameters, overfitting, PEFT, LoRA, QLoRA, SFT, instruction tuning, evaluation, deployment, and scenario-based interview questions.

1. What is Fine-Tuning?

Fine-tuning is the process of taking a pre-trained model and continuing training on a smaller, task-specific dataset so that the model becomes better suited to a particular task, domain, behavior, or output style.



The key distinction is that the model is not trained from scratch. It starts with useful representations already learned during pre-training.

2. Why Do We Fine-Tune?

- Task specialization: improve performance on a particular task.
- Instruction following: make the model follow a desired input/output pattern.
- Style and format: consistently produce a particular structure, tone, or schema.
- Domain adaptation: adapt to specialized terminology and patterns.
- Classification: adapt an encoder or generative model to classify inputs.
- Behavior adaptation: make desired responses more consistent.

Fine-tuning is usually not the first choice simply because you have a large document collection. If the requirement is to retrieve changing/private facts from documents, RAG is often a better fit.

3. Pre-Training vs Fine-Tuning vs Prompting vs RAG

Approach	What changes?	Main purpose
Pre-training	Model parameters from initial training	Learn broad/general capabilities
Fine-tuning	Model parameters or trainable adapters	Specialize behavior/task/domain
Prompt engineering	No model parameters	Change instructions at inference time
RAG	Usually no model parameters	Supply external/current knowledge at inference time

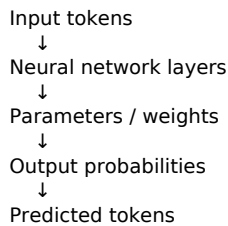
4. A Simple Analogy

Imagine a student who has already completed school and college. That is the pre-trained model. If you now train the student specifically for DSA interviews using many examples, that is analogous to fine-tuning.

General education → Pre-training
Interview specialization → Fine-tuning
A written instruction → Prompt engineering
Looking up a current textbook → RAG

5. What Is Inside an LLM?

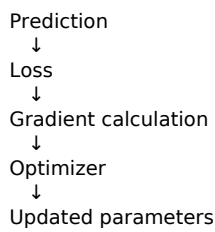
An LLM is a neural network containing many learned numerical parameters, commonly called weights. During training, these parameters are adjusted so the model becomes better at its objective.



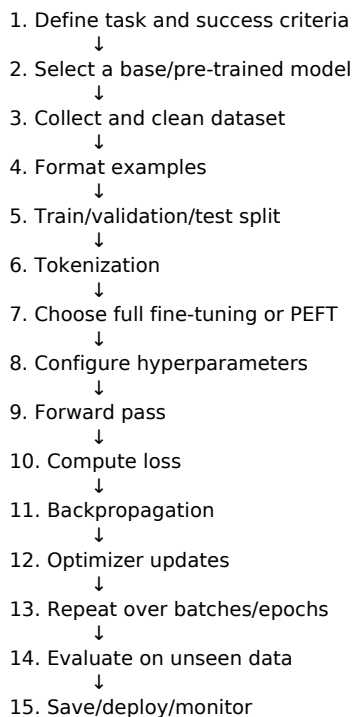
Fine-tuning modifies the model's learned parameters—or a small set of additional trainable parameters in PEFT—based on your new dataset.

6. Parameters, Weights and Gradients

- Parameter/weight: a learned numerical value inside the model.
- Gradient: indicates how changing a parameter would affect the loss.
- Loss: a numerical measure of prediction error.
- Optimizer: uses gradients to update trainable parameters.



7. End-to-End Fine-Tuning Pipeline



8. Step 1 — Define the Task

Before training, clearly define what the model should improve at. Examples include summarization, classification, extraction, instruction following, domain-specific generation, or a strict output format.

- Define inputs and desired outputs.
- Define what 'good' means using measurable evaluation criteria.
- Identify constraints such as latency, cost, privacy, and deployment hardware.

9. Step 2 — Choose the Base Model

Choose a model appropriate for the task, license, language, context length, hardware, and expected quality. A smaller model may be preferable when latency or deployment cost is important.

- Generative tasks → a suitable causal language model.
- Classification/representation tasks → encoder models can be appropriate.
- Check tokenizer compatibility, supported context length, quantization options, and license.

10. Step 3 — Dataset Creation

The dataset is often the most important practical component. A fine-tuning dataset should be representative of the behavior you want at inference time.

Example instruction dataset

Input:
"Summarize this medical report..."

Target:
"Summary:
- Finding: ...
- Severity: ...
- Recommendation: ..."

- Correct and high-quality examples.
- Representative of real user inputs.
- Enough variation to avoid simple memorization.
- Consistent formatting.
- Remove duplicates and contradictory examples.
- Remove sensitive information unless it is intentionally handled under appropriate controls.

11. Dataset Quality vs Dataset Size

More examples are not automatically better. Low-quality or contradictory data can teach undesirable behavior. A smaller, carefully curated dataset can outperform a larger noisy dataset.

10,000 high-quality examples
can be better than
100,000 noisy examples

12. Train / Validation / Test Split

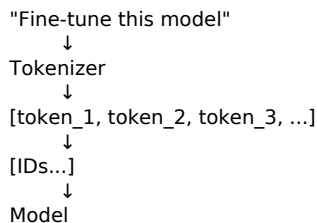
The training set is used for parameter updates. Validation data is used during development to make decisions and monitor generalization. A held-out test set is used for final evaluation.

Dataset
├── Train → learn
├── Validation → tune/monitor
└── Test → final unbiased evaluation

The exact proportions depend on the dataset and project. Avoid leaking near-duplicates across splits.

13. Tokenization

Language models process token IDs rather than raw text. A tokenizer converts text into tokens and then into numerical IDs.



Tokenization is model-specific. One word can become one token, multiple tokens, or part of a token depending on the tokenizer.

14. Sequence Length and Padding

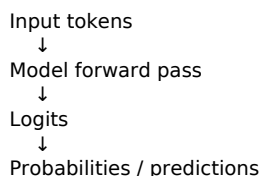
Training examples may have different lengths. Batching often requires them to be padded to a compatible shape. The model uses attention masks so padding tokens do not behave like meaningful input.

- Max sequence length: limits how much tokenized content is processed per example.
- Padding: makes examples in a batch align in length.
- Truncation: cuts examples that exceed a selected maximum length.

Longer sequences increase memory and compute requirements, often substantially.

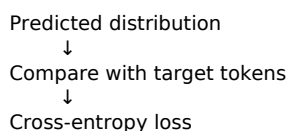
15. Forward Pass

During a forward pass, tokenized input is processed through the model and the model produces logits/probabilities for possible next tokens or task outputs.



16. Loss Function

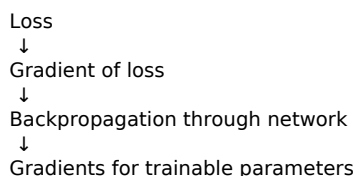
The loss measures how far the model's predictions are from the training target. For causal language modeling, cross-entropy loss is commonly used.



The exact loss depends on the task. Classification, sequence labeling, and generative tasks can use different objectives.

17. Backpropagation

Backpropagation computes gradients of the loss with respect to trainable parameters. These gradients tell the optimizer how parameters should be adjusted to reduce the loss.



18. Optimizers

An optimizer updates trainable parameters using gradients. Adam and AdamW are widely used in transformer training/fine-tuning.

```

old parameter
+
gradient information
↓
optimizer
↓
new parameter

```

AdamW is particularly common because it handles weight decay in a way designed for better optimization behavior than naively mixing weight decay into Adam's gradient update.

19. Learning Rate

Learning rate controls the scale of parameter updates.

- Too high → training can become unstable or destroy useful pretrained behavior.
- Too low → training can be extremely slow or fail to adapt enough.

Learning rate is one of the most important hyperparameters to tune.

20. Batch Size

Batch size is the number of examples processed together before an optimizer update, subject to gradient accumulation and implementation details.

- Larger batches generally require more memory.
- Smaller batches use less memory but may have noisier gradient estimates.
- Gradient accumulation allows multiple smaller batches to contribute to an effective larger batch before updating parameters.

21. Epoch, Step and Iteration

Epoch: one complete pass over the training dataset. Training step: typically one optimizer update. The exact relationship depends on batch size, dataset size, and gradient accumulation.

```

Dataset
↓
Batch 1 → update
Batch 2 → update
...
Batch N → update
↓
One epoch completed

```

22. Overfitting

Overfitting occurs when a model becomes too specialized to training examples and generalizes poorly to unseen data.

```

Training loss ↓ ↓ ↓
Validation loss ↓ then ↑
      ↑
    possible overfitting

```

- Use validation data.
- Reduce excessive training.
- Use appropriate regularization.
- Improve dataset diversity.
- Consider early stopping when appropriate.

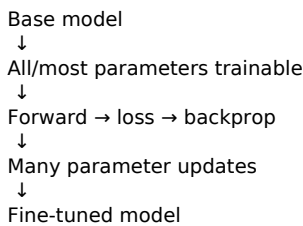
23. Catastrophic Forgetting

During adaptation, a model can lose some capabilities or general knowledge it previously had, especially when fine-tuning strongly on a narrow distribution.

- Use a suitable learning rate.
- Use high-quality and diverse data.
- Avoid unnecessary training for too many epochs.
- Consider PEFT when appropriate.
- Evaluate both the target task and important general capabilities.

24. Full Fine-Tuning

In full fine-tuning, most or all model parameters that are designated trainable are updated.



For large models, this can be expensive in GPU memory, compute, training time, and checkpoint storage.

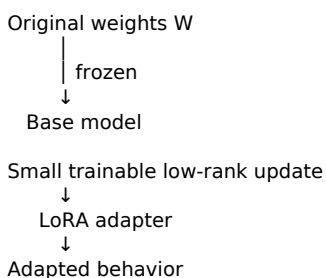
25. PEFT — Parameter-Efficient Fine-Tuning

PEFT reduces the number of parameters that must be trained. The original model can remain frozen while a small set of trainable parameters/adapters learns the task.

- Lower memory requirements.
- Smaller trainable artifacts.
- Can make adaptation of larger models more practical.
- Allows task-specific adapters to be managed separately from a base model.

26. LoRA

LoRA stands for Low-Rank Adaptation. Instead of directly learning a full update to selected large weight matrices, LoRA represents the adaptation using low-rank trainable matrices.



The exact mathematical formulation can be expressed conceptually as an update to a base weight matrix using low-rank matrices A and B, often written as $\Delta W = B \cdot A$ (with scaling applied by the method).

27. What Does LoRA Rank r Mean?

The rank controls the dimensionality/capacity of the low-rank update. A higher rank generally means more trainable parameters and potentially greater adaptation capacity, while increasing resource usage.

Lower r → fewer adapter parameters
Higher r → more adapter capacity

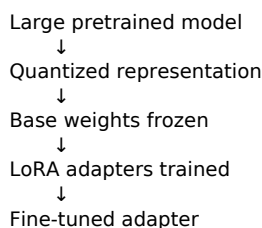
28. LoRA Alpha and Dropout

Alpha is commonly used as a scaling factor for the LoRA update. Dropout can be applied within the adapter path as a regularization technique. Exact configuration depends on the implementation.

Do not memorize arbitrary values as universal best settings; tune them for the model, dataset, and hardware.

29. QLoRA

QLoRA combines quantized loading of the base model with LoRA-based adaptation. The main goal is to reduce memory usage while retaining a practical fine-tuning workflow.



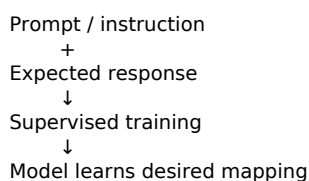
A common implementation uses low-bit quantization for the base model and higher-precision computation for adapter training. Exact quantization details depend on the stack and hardware.

30. LoRA vs QLoRA

	LoRA	QLoRA
Base model	Often loaded in regular precision	Loaded in quantized/low-bit form
Trainable part	LoRA adapters	LoRA adapters
Main benefit	Fewer trainable parameters	Fewer trainable parameters + lower base-model memory
Use case	PEFT with suitable hardware	Large models under tighter GPU memory constraints

31. Supervised Fine-Tuning (SFT)

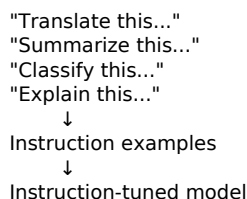
SFT means Supervised Fine-Tuning. The model is trained on examples where a desired target response is provided.



SFT is a common stage in instruction-tuned language model pipelines.

32. Instruction Tuning

Instruction tuning is fine-tuning on examples that teach a model to respond appropriately to natural-language instructions across tasks.



33. Causal LM Fine-Tuning

For a causal language model, the training objective commonly teaches the model to predict the next token based on previous tokens. During supervised instruction tuning, the training sequence can contain both user/instruction content and target assistant content, with the loss often focused on target portions depending on the training setup.

Tokens: The cat sat on the mat
Targets: cat sat on the mat ...
Predict next token from previous context

34. Loss Masking

In instruction tuning, you may not want every token in the prompt to contribute equally to the loss. Some training setups mask the loss on instruction/user tokens and calculate it primarily on the desired assistant response.

User tokens: [MASKED from loss]
Assistant tokens: [LOSS ACTIVE]

This helps align the training objective with learning the desired response, but the exact setup is implementation-dependent.

35. Data Leakage

Data leakage occurs when information from evaluation data influences training or model selection. This can make reported performance look much better than real-world performance.

- Keep test data isolated.
- Check for duplicates/near-duplicates across splits.
- Avoid using test examples to tune hyperparameters.
- Be careful with public benchmark contamination.

36. Hyperparameters You Should Know

Hyperparameter	Meaning
Learning rate	Scale of parameter updates
Batch size	Examples processed together
Epochs	Passes over training dataset
Weight decay	Regularization affecting parameter magnitudes
Warmup	Gradually increases learning rate at the start
Max sequence length	Maximum tokens processed per example
Gradient accumulation	Accumulates gradients over multiple batches
LoRA rank (r)	Capacity/dimensionality of low-rank adapter
LoRA alpha	Scaling factor for LoRA update
LoRA dropout	Adapter regularization setting

37. Evaluation

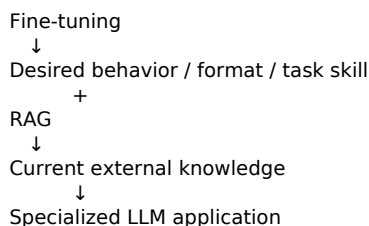
Do not judge a fine-tuned model only by training loss.

- Task metrics: accuracy, F1, precision/recall, BLEU/ROUGE where appropriate, exact match, etc.
- Generative evaluation: factuality, instruction adherence, helpfulness, style, structured-output validity.
- Human evaluation: useful for subjective quality and high-value applications.
- Regression testing: verify that important old capabilities did not degrade.

Choose metrics based on the actual task. There is no single universal LLM fine-tuning metric.

38. Fine-Tuning and RAG Together

They solve different problems and can be combined.



Example: fine-tune a support model to consistently produce a structured support response, while RAG retrieves the company's latest policies and product information.

39. Fine-Tuning vs RAG — Interview Table

Question	Prefer RAG	Prefer Fine-Tuning
Need current/private searchable facts?	✓	Usually not primary solution
Need consistent style/format?	Possible with prompting	✓
Need task-specific behavior?	Not primary solution	✓
Knowledge changes frequently?	✓	Usually inefficient
Need external citations?	✓	Not inherent
Need model to learn a repeated output pattern	Possible	✓

40. Practical Hugging Face-Style Workflow

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer
)

model_name = "your-base-model"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Your cleaned and formatted dataset
train_dataset = ...
eval_dataset = ...

args = TrainingArguments(
    output_dir="/output",
    num_train_epochs=3,
    learning_rate=2e-5,
    per_device_train_batch_size=4,
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)

trainer.train()
```

This is a learning template rather than a universal production configuration. Real projects need model-specific tokenizer handling, data collators, evaluation, checkpointing, memory optimization, and hardware-aware settings.

41. LoRA-Style Implementation Concept

```

from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"]
)

model = get_peft_model(model, config)

model.print_trainable_parameters()

```

The critical concept is that the base model can be frozen while the LoRA adapter becomes trainable. Target module names are model-architecture dependent; do not assume q_proj/v_proj is universal.

42. What Happens During One Training Step?

```

Batch of examples
↓
Tokenizer / data collator
↓
Forward pass
↓
Logits
↓
Loss
↓
Backward pass
↓
Gradients
↓
Optimizer
↓
Update trainable parameters
↓
Next batch

```

43. What Happens in Full Fine-Tuning vs LoRA?

Stage	Full fine-tuning	LoRA
Base weights	Trainable	Usually frozen
Extra adapter	No	Yes
Trainable parameters	Large	Small
GPU memory	Higher	Lower
Checkpoint size	Can be large	Adapter can be much smaller
Deployment	Fine-tuned model	Base model + adapter, or merged form where supported

44. Quantization

Quantization represents model values using lower numerical precision, which can reduce memory usage and sometimes improve inference efficiency. In QLoRA-style workflows, quantization is primarily used to make the frozen base model cheaper to load while adapters are trained.

- Quantization is different from fine-tuning.
- LoRA is an adaptation method.
- QLoRA combines quantized base-model loading with LoRA training.

45. Deployment Choices

After training, decide how the adapted model will be served.

- Serve the base model with the adapter.

- Merge adapters into base weights when supported and appropriate.
- Quantize for lower-memory inference.
- Use a model-serving stack compatible with the chosen architecture.

Deployment should be tested for quality, latency, throughput, memory use, and cost—not just model accuracy.

46. Monitoring After Deployment

- Track input distribution changes.
- Track task quality and user feedback.
- Monitor latency and token usage.
- Detect safety or policy failures.
- Run regression tests after model/data changes.
- Monitor whether real-world inputs differ significantly from fine-tuning data.

47. When Should You NOT Fine-Tune?

- When the only requirement is access to frequently changing documents.
- When a good prompt already achieves the required behavior.
- When you do not have enough representative high-quality examples.
- When the task is better solved with deterministic code or a database.
- When the training/deployment cost is unjustified by the expected quality gain.

A good engineer chooses fine-tuning because evaluation shows it is useful—not because fine-tuning is fashionable.

48. Common Mistakes

- Fine-tuning on unclean or contradictory data.
- Using the test set to tune the model.
- Training for too many epochs.
- Using an unnecessarily high learning rate.
- Ignoring catastrophic forgetting.
- Assuming more Top-K/RAG context or more training data is always better.
- Confusing RAG with fine-tuning.
- Assuming LoRA target modules are identical for every model.
- Evaluating only on training loss.
- Deploying without checking license, privacy, security, latency, and cost.

49. Interview Questions — Basic

Q: What is fine-tuning?

Answer: Fine-tuning is continued training of a pre-trained model on task- or domain-specific examples to adapt its behavior or performance.

Q: Does fine-tuning train a model from scratch?

Answer: No. It starts from a pre-trained model and continues optimization from its learned parameters.

Q: Why do we fine-tune?

Answer: To specialize a model for a task, domain, behavior, style, or output format when prompting alone is insufficient.

Q: What is the difference between pre-training and fine-tuning?

Answer: Pre-training builds broad capabilities from very large datasets; fine-tuning adapts an already trained model to a narrower objective.

Q: What is an epoch?

Answer: One pass through the training dataset.

Q: What is learning rate?

Answer: A hyperparameter controlling the scale of parameter updates.

Q: What is loss?

Answer: A numerical measure of how poorly the model's prediction matches the training target.

Q: What is overfitting?

Answer: When training performance improves while generalization to unseen data worsens.

Q: What is PEFT?

Answer: Parameter-Efficient Fine-Tuning, where only a small portion of parameters or additional adapter parameters are trained.

Q: What is LoRA?

Answer: A PEFT method that learns low-rank updates while usually keeping the base model weights frozen.

50. Interview Questions — Intermediate

Q: Why is dataset quality important?

Answer: Because the model learns from the examples. Incorrect, noisy, contradictory, or unrepresentative examples can produce undesirable behavior.

Q: What is catastrophic forgetting?

Answer: Loss of some previously learned capabilities while adapting strongly to a new narrow dataset.

Q: Why use validation data?

Answer: To monitor generalization and make development decisions without repeatedly optimizing against the final test set.

Q: Why is full fine-tuning expensive?

Answer: Large models have many trainable parameters, so optimizer states, gradients, activations, and parameter updates require substantial memory and compute.

Q: Why is LoRA cheaper?

Answer: The base weights are generally frozen and only small low-rank adapter parameters are trained, reducing trainable state and memory requirements.

Q: What is QLoRA?

Answer: A method that combines quantized base-model loading with LoRA adapters to make parameter-efficient fine-tuning more memory-efficient.

Q: What is SFT?

Answer: Supervised Fine-Tuning, where the model learns from input/target examples with known desired outputs.

Q: What is instruction tuning?

Answer: Fine-tuning on instruction-response examples so the model becomes better at following natural-language instructions across tasks.

Q: What is gradient accumulation?

Answer: Accumulating gradients over several smaller batches before performing an optimizer update to achieve a larger effective batch size.

51. Interview Questions — Advanced

Q: How would you decide between full fine-tuning and LoRA?

Answer: I would compare the required adaptation capacity with hardware and deployment constraints. If full adaptation is justified and resources allow it, full fine-tuning is possible; otherwise I would evaluate PEFT such as LoRA. I would make the decision based on measured task performance and resource cost.

Q: Can LoRA change the model's behavior?

Answer: Yes. The trained adapter changes the effective model computation so the combined base model plus adapter produces different behavior.

Q: Can RAG and fine-tuning be combined?

Answer: Yes. Fine-tuning can specialize behavior while RAG supplies current or external knowledge at inference time.

Q: Does fine-tuning guarantee factual accuracy?

Answer: No. Fine-tuning can improve task behavior, but it does not guarantee that generated facts are correct or current.

Q: How would you debug poor fine-tuning results?

Answer: I would inspect dataset quality and formatting, tokenizer/truncation, train/validation splits, loss behavior, learning rate, batch size, number of epochs, target modules for PEFT, evaluation methodology, and whether the base model is appropriate.

Q: What if training loss is low but real-world performance is poor?

Answer: I would suspect overfitting, data mismatch, leakage, poor evaluation design, or a mismatch between the training objective and the real task.

Q: How would you prevent overfitting?

Answer: Use representative data, monitor validation performance, tune training duration and learning rate, apply appropriate regularization, and evaluate on a truly held-out dataset.

Q: How would you fine-tune a model with limited GPU memory?

Answer: I would consider PEFT such as LoRA and potentially QLoRA, use memory-efficient batching/gradient accumulation, sequence-length controls, checkpointing where appropriate, and hardware-compatible precision/quantization.

52. Scenario Questions

Q: A company has 500 GB of changing internal documents. Fine-tune or RAG?

Answer: I would generally start with RAG because the primary need is retrieving current external knowledge. Fine-tuning may still be useful later if the desired response behavior needs specialization.

Q: A chatbot always gives correct information but uses inconsistent output formatting. What would you try?

Answer: I would first use a strong prompt and structured output constraints. If evaluation shows persistent behavior that warrants learned adaptation and I have representative examples, I would consider supervised fine-tuning.

Q: The model performs well on training data and badly on new examples. What is likely happening?

Answer: Overfitting or train/deployment distribution mismatch are likely possibilities. I would inspect validation/test performance and data diversity.

Q: You have only 500 examples. Can you fine-tune?

Answer: Possibly, but the answer depends on task complexity and example quality. I would consider whether prompting or PEFT is sufficient, carefully evaluate generalization, and avoid assuming that a small dataset will support broad adaptation.

Q: Fine-tuning improved the target task but broke general capabilities. What happened?

Answer: This may indicate catastrophic forgetting or overly aggressive/narrow adaptation. I would review learning rate, epochs, dataset composition, and consider PEFT or more diverse training data.

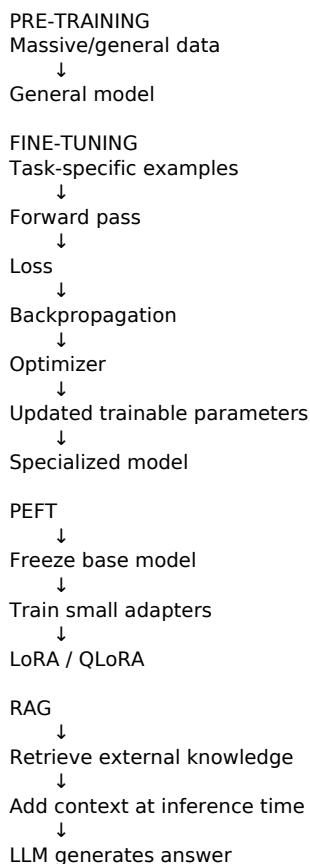
53. Strong 30-Second Interview Answer

“Fine-tuning is the process of adapting a pre-trained model using task- or domain-specific examples. We first define the task, select a suitable base model, prepare and clean representative training data, tokenize it, split it into train and validation/test sets, and then train using an objective such as cross-entropy. During each training step the model performs a forward pass, computes loss, backpropagates gradients, and an optimizer updates the trainable parameters. For large models, I would consider parameter-efficient methods such as LoRA or QLoRA to reduce memory and compute. Finally, I would evaluate on unseen data and monitor both target performance and regressions in existing capabilities.”

54. Strong RAG vs Fine-Tuning Answer

“I see RAG and fine-tuning as solving different problems. RAG is primarily about giving the model access to external, private, or changing information at inference time, while fine-tuning is about adapting the model's learned behavior or task performance. If a company's policies change frequently, I would usually use RAG rather than repeatedly fine-tuning. If I need consistent response style, structure, or specialized task behavior, I would evaluate fine-tuning. In some systems, I would combine both.”

55. One-Page Mental Model



56. Final Cheat Sheet

Concept	One-line memory
Pre-training	Build general model capabilities.
Fine-tuning	Adapt a pre-trained model using task-specific examples.
SFT	Learn from supervised input → target examples.
Instruction tuning	Teach better instruction following.
Parameter	Learned numerical value in a model.
Gradient	Direction/sensitivity used to update trainable parameters.
Loss	Measures prediction error.
Optimizer	Uses gradients to update parameters.
Epoch	One pass through training data.
Overfitting	Good training performance, poor unseen-data generalization.
Catastrophic forgetting	Loss of previous capabilities during adaptation.
PEFT	Train a small parameter subset/additional adapters.
LoRA	Low-rank trainable adaptation with frozen base weights.
QLoRA	Quantized base model + LoRA adaptation.
RAG	Retrieve external context instead of changing model weights.
Prompting	Change instructions without training model weights.

Final interview rule: Do not present fine-tuning as the solution to every LLM problem. Start with the requirement, distinguish knowledge retrieval from behavior adaptation, choose prompting/RAG/fine-tuning/PEFT based on evidence, and explain how you would evaluate the result.